

实验二：分布式训练实战 (DeepSpeed/PyTorch)

1. 实验目标

在大模型时代，单卡显存往往无法支撑训练需求。本实验旨在通过实践，让学生掌握分布式训练的核心技术，包括**数据并行 (DP/DDP)**、**ZeRO 优化器策略**以及**混合精度训练**。学生需将实验一中的 Transformer 模型迁移至分布式环境。

2. 实验核心任务

- **环境构建与模拟**：配置分布式训练环境。在单卡环境下，利用 PyTorch 的多进程机制 (`torch.distributed`) 或 DeepSpeed 模拟多 GPU 并行行为。
- **数据分发 (Distributed Sampler)**：实现数据集在不同进程间的非重叠划分，确保每个“虚拟显卡”训练不同的数据分片。
- **ZeRO 优化器配置**：调用 DeepSpeed，配置 `zero_optimization` 策略 (至少实现 ZeRO-1 或 ZeRO-2)，观察显存占用的变化。
- **混合精度加速**：启用 **FP16 或 BF16** 进行计算，对比其与 FP32 在训练速度和收敛稳定性上的差异。

3. 实验指南

第一步：分布式环境初始化

- **任务**：在代码中添加分布式启动逻辑。
- **关键点**：设置 `rank` (进程编号)、`world_size` (总进程数) 和 `master_addr`。使用 `dist.init_process_group` 初始化通讯后端 (建议使用 `nccl` 或 `gloo`)。

第二步：模型与优化器包装

- **任务**：将实验一的医学分类模型封装进分布式框架。
- **DeepSpeed 路径**：编写 `ds_config.json` 配置文件，设置 `train_batch_size`、`steps_per_print` 以及核心的 `zero_stage`。
- **DDP 路径**：使用 `torch.nn.parallel.DistributedDataParallel` 包装模型，并处理多卡间的梯度同步。

第三步：显存与效率观测 (实验重点)

这是本实验的核心教学点。要求学生记录并对比以下三种配置下的性能：

1. **Baseline**：单卡 FP32 训练。
 2. **配置 A**：多卡 DDP + FP16。
 3. **配置 B**：多卡 DeepSpeed + ZeRO-2 + 梯度累积 (Gradient Accumulation)。
- **观察指标**：单步迭代耗时 (Throughput)、峰值显存占用 (Max Memory Allocated)。

第四步：模型验证与评估

- **逻辑一致性**：确保分布式训练后的模型在 test.json 上的准确率与单卡训练一致 (验证 All-Reduce 梯度同步是否正确)。
- **Loss 曲线图**：绘制不同进程数 (如 2 进程 vs 4 进程) 下的 Loss 下降曲线。

4. 实验报告要求

1. **代码实现**：提交分布式训练脚本。必须包含 **DeepSpeed 配置文件 (ds_config.json)** 或 **DDP 初始化代码段**。
2. **配置对比表**：整理一份表格，记录不同并发数、不同 ZeRO 阶段下的**显存占用**和**训练吞吐量 (tokens/sec)**。
3. **技术分析**：
 - a. 简述 **ZeRO-2** 是如何通过消除冗余来节省显存的。
 - b. 解释为什么在分布式训练中需要使用 DistributedSampler？如果不使用会有什么后果？
4. **实验报告提交**：发送至老师邮箱：sdzhao@ir.hit.edu.cn

5. 关键代码示例 (DeepSpeed 配置参考)

JSON

```
1  {
2    "train_batch_size": 32,
3    "steps_per_print": 10,
4    "optimizer": {
5      "type": "AdamW",
6      "params": { "lr": 0.001, "weight_decay": 0.1 }
7    },
8    "fp16": { "enabled": true },
9    "zero_optimization": {
10     "stage": 2,
11     "allgather_partitions": true,
12     "reduce_scatter": true,
13     "overlap_comm": true,
14     "contiguous_gradients": true
15   }
16 }
```